Lecture 10: Automatic differentiation and when it fails

COMP 521 • Probability, Optimization, and Scientific Discovery • Sections 5.1–5.4

This handout is one part of the full COMP 521 notes. Section, equation, figure and listing numbers match the full notes, and references to anything outside this handout point there. The notation table is in "How to use these notes" (Lecture 1).

5 Differentiable and trustworthy scientific computing

5.1 Automatic differentiation: JVPs and VJPs

Automatic differentiation (AD) applies the chain rule to the elementary operations a program actually executes [1]. It is exact up to floating-point rounding for the program as written. It differs from finite differences, which perturb inputs and approximate, and from symbolic differentiation, which manipulates formulas and can blow up in size.

For $\mathcal{G} : \mathbb{R}^p \to \mathbb{R}^m$ with Jacobian $\mathbf{J}$ there are two primitive operations:

- a **Jacobian–vector product** (JVP) $\mathbf{Jv}$: how all outputs change along one input direction $\mathbf{v}$. Forward mode computes it by carrying a tangent alongside each value through the program ("dual numbers", $x + \epsilon\dot{x}$ with $\epsilon^2 = 0$).
- a **vector–Jacobian product** (VJP) $\mathbf{u}^\mathsf{T}\mathbf{J}$: how one weighted combination of outputs depends on all inputs. Reverse mode computes it by running the program forward, storing intermediates, then propagating sensitivities backward.

Task	Forward mode	Reverse mode
gradient of a scalar loss ($m = 1$)	p passes	1 pass (a few times the cost of $\mathcal{G}$)
full Jacobian	p passes	m passes
memory	like $\mathcal{G}$	grows with the length of the computation

A scalar loss of many parameters favours reverse mode; a Jacobian with few inputs and many outputs (5 physics parameters, 2000 points on a track) favours forward mode. Reverse mode's memory cost is the practical obstacle for long stellar evolutions, mitigated by checkpointing (recomputing intermediates instead of storing them) and by the implicit methods of Section 5.2. In JAX: `jax.jvp`, `jax.vjp`, `jax.grad`, `jax.jacfwd`, `jax.jacrev`, `jax.hessian`, `jax.linearize`, and `jax.vmap` to batch any of them.

Key idea

AD returns the derivative of *the program*. Whether that equals the derivative of *the physics* is a separate question, answered by Sections 5.3 and 5.4.

5.2 Differentiating through an implicit solve: the adjoint method

Stellar structure defines the state implicitly, $\mathbf{R}(\mathbf{s}, \boldsymbol{\theta}) = \mathbf{0}$ (Eq. (15)). Differentiating the identity $\mathbf{R}(\mathbf{s}(\boldsymbol{\theta}), \boldsymbol{\theta}) = \mathbf{0}$ gives

$$\mathbf{R}_s \frac{\mathrm{d}\mathbf{s}}{\mathrm{d}\boldsymbol{\theta}} + \mathbf{R}_\theta = \mathbf{0} \quad \Longrightarrow \quad \frac{\mathrm{d}\mathbf{s}}{\mathrm{d}\boldsymbol{\theta}} = -\mathbf{R}_s^{-1}\mathbf{R}_\theta, \tag{39}$$

with $\mathbf{R}_s = \partial \mathbf{R}/\partial \mathbf{s}$ (the Jacobian the Newton solver already factors) and $\mathbf{R}_\theta = \partial \mathbf{R}/\partial \boldsymbol{\theta}$. This is the *implicit function theorem*: no derivative of the Newton iterations themselves is needed, only of the residual at the solution. Forming $\mathrm{d}\mathbf{s}/\mathrm{d}\boldsymbol{\theta}$ costs p linear solves.

Usually we want the gradient of a scalar $\mathcal{L}(\mathbf{s}(\boldsymbol{\theta}), \boldsymbol{\theta})$. By the chain rule,

$$\frac{\mathrm{d}\mathcal{L}}{\mathrm{d}\boldsymbol{\theta}} = \frac{\partial \mathcal{L}}{\partial \boldsymbol{\theta}} + \frac{\partial \mathcal{L}}{\partial \mathbf{s}} \frac{\mathrm{d}\mathbf{s}}{\mathrm{d}\boldsymbol{\theta}} = \frac{\partial \mathcal{L}}{\partial \boldsymbol{\theta}} - \underbrace{\frac{\partial \mathcal{L}}{\partial \mathbf{s}} \mathbf{R}_s^{-1}}_{\boldsymbol{\lambda}^\mathsf{T}} \mathbf{R}_\theta.$$

Grouping the product the other way defines the *adjoint* vector $\boldsymbol{\lambda}$:

$$\mathbf{R}_s^\mathsf{T} \boldsymbol{\lambda} = \left(\frac{\partial \mathcal{L}}{\partial \mathbf{s}}\right)^\mathsf{T}, \qquad \frac{\mathrm{d}\mathcal{L}}{\mathrm{d}\boldsymbol{\theta}} = \frac{\partial \mathcal{L}}{\partial \boldsymbol{\theta}} - \boldsymbol{\lambda}^\mathsf{T} \mathbf{R}_\theta. \tag{40}$$

One transposed linear solve gives the whole gradient, however many parameters there are. The adjoint method is reverse-mode AD applied at the level of the solver, and it is how gradients of PDE- and ODE-constrained problems are computed in practice.

ex09_implicit_adjoint.py -- four ways to differentiate through a nonlinear solve

```python
"""Gradients through an implicit (nonlinear) solve: unrolled AD, forward
sensitivities, the adjoint method, and jax.lax.custom_root.

Toy 'structure' problem: find s in R^n with  R(s, theta) = A s + s**3 - theta = 0,
where A is a symmetric positive-definite discrete Laplacian (so the solution is unique).
Scalar objective: Lfun(s) = 1/2 ||s - s_obs||^2.
"""
import jax
import jax.numpy as jnp

jax.config.update("jax_enable_x64", True)

n = 50
A = (jnp.diag(2.0 * jnp.ones(n)) - jnp.diag(jnp.ones(n - 1), 1)
     - jnp.diag(jnp.ones(n - 1), -1)) * n**2 / 100.0
x = jnp.linspace(0, 1, n)
theta = 1.0 + jnp.sin(2 * jnp.pi * x)  # 'physics parameters' (here: a source profile)
s_obs = 0.5 * jnp.ones(n)


def R(s, theta):
    return A @ s + s**3 - theta


def newton_solve(theta, iters=30):
    s = jnp.zeros(n)
    for _ in range(iters):  # Python loop: jax.grad will unroll and differentiate it
        s = s - jnp.linalg.solve(jax.jacfwd(R)(s, theta), R(s, theta))
    return s


def objective(s):
    return 0.5 * jnp.sum((s - s_obs) ** 2)


# (1) Differentiate through every Newton iteration (memory grows with iterations)
g_unrolled = jax.grad(lambda th: objective(newton_solve(th)))(theta)

```

```python
# At the solution, form the Jacobians of the residual
s_star = newton_solve(theta)
R_s = jax.jacfwd(R, argnums=0)(s_star, theta)  # n x n
R_th = jax.jacfwd(R, argnums=1)(s_star, theta)  # n x n (here: -I)
dL_ds = jax.grad(objective)(s_star)

# (2) Forward sensitivities: ds/dtheta = -R_s^{-1} R_th  -> n linear solves
ds_dth = -jnp.linalg.solve(R_s, R_th)
g_forward = ds_dth.T @ dL_ds

# (3) Adjoint: ONE transposed solve, R_s^T lam = dL/ds, then g = -lam^T R_th
lam = jnp.linalg.solve(R_s.T, dL_ds)
g_adjoint = -R_th.T @ lam


# (4) Let JAX apply the implicit function theorem for us
def solve_implicit(theta):
    newton = lambda f, s0: newton_solve(theta)  # the forward solve itself
    lin = lambda g, y: jnp.linalg.solve(jax.jacfwd(g)(y), y)  # tangent solve
    return jax.lax.custom_root(lambda s: R(s, theta), jnp.zeros(n), newton, lin)


g_custom = jax.grad(lambda th: objective(solve_implicit(th)))(theta)

print(f"residual norm at solution : {jnp.linalg.norm(R(s_star, theta)):.1e}")
for name, g in [("forward sens.", g_forward), ("adjoint", g_adjoint),
                ("custom_root", g_custom)]:
    print(f"max |g_{name} - g_unrolled| = {jnp.max(jnp.abs(g - g_unrolled)):.1e}")
```

Output (executed for these notes):

```
residual norm at solution : 3.4e-14
max |g_forward sens. - g_unrolled| = 8.3e-17
max |g_adjoint - g_unrolled| = 5.6e-17
max |g_custom_root - g_unrolled| = 8.3e-17
```

All four agree to rounding error. They differ in cost: unrolling stores every Newton iterate; forward sensitivities need p solves; the adjoint needs one; `jax.lax.custom_root` applies the implicit function theorem automatically. Libraries such as Optimistix (implicit adjoints for root finds and minimizations) and Diffrax (adjoints for ODE solves) package the same idea; see [2] for a general treatment of implicit differentiation in JAX.

A singular $\mathbf{R}_s$ is information

Equations (39) and (40) need $\mathbf{R}_s$ to be invertible. Near a fold or bifurcation of the solution family (for instance, where two structure solutions merge) $\mathbf{R}_s$ becomes singular and sensitivities diverge. That is the mathematics reporting a qualitative change in the physics, not a numerical nuisance to be regularized away.

5.3 Verifying a derivative

Every gradient used for inference should be checked against an independent method at representative points, especially near regime changes.

Finite differences and their error budget. The forward difference $[f(x+h)-f(x)]/h$ has truncation error $\approx \frac{1}{2}|f''|h$ and rounding error $\approx \epsilon_{\text{mach}}|f|/h$; their sum is smallest at $h \sim \sqrt{\epsilon_{\text{mach}}} \approx 10^{-8}$, giving relative accuracy about 10^{-8}. The central difference $[f(x+h)-f(x-h)]/2h$ has truncation error $O(h^2)$, so the optimum moves to $h \sim \epsilon_{\text{mach}}^{1/3} \approx 6 \times 10^{-6}$ and the accuracy to about 10^{-10}. Smaller is not better: below the optimum, rounding wins.

The complex step. For a real-analytic f implemented with complex-safe operations, $f(x+ih) = f(x)+ihf'(x)+O(h^2)$, so $f'(x) \approx \operatorname{Im} f(x+ih)/h$ involves no subtraction and is accurate to machine precision for any tiny h, even 10^{-30} [3]. It fails for non-analytic operations (`abs`, comparisons, `max`).

In many dimensions, check directional derivatives: for random $\mathbf{v}$, compare the JVP $\mathbf{Jv}$ with $[\mathcal{G}(\boldsymbol{\theta}+h\mathbf{v})-\mathcal{G}(\boldsymbol{\theta}-h\mathbf{v})]/2h$, and check reverse against forward mode with the identity $\mathbf{u}^\mathsf{T}(\mathbf{Jv}) = (\mathbf{u}^\mathsf{T}\mathbf{J})\mathbf{v}$. `jax.test_util.check_grads` automates these comparisons.

ex08_gradient_checks.py -- derivative checks, and a confidently wrong gradient

```python
"""Verifying derivatives: finite differences, complex step, check_grads,
and a case where AD returns a confidently wrong answer (an event threshold).
"""
import numpy as np
import jax
import jax.numpy as jnp
from jax.test_util import check_grads

jax.config.update("jax_enable_x64", True)


# --- Part A: a smooth function (dimensionless Planck-like occupation number) ---
def f(T):
    return 1.0 / (jnp.exp(1.0 / T) - 1.0)


def df_exact(T):
    e = np.exp(1.0 / T)
    return e / (T**2 * (e - 1.0) ** 2)


T0 = 0.3
print("Part A: d/dT [1/(exp(1/T)-1)] at T = 0.3")
print(f"  exact      = {df_exact(T0):.15e}")
print(f"  jax.grad   = {jax.grad(f)(T0):.15e}")
for h in [1e-2, 1e-5, 1e-8, 1e-11]:
    fwd = (f(T0 + h) - f(T0)) / h
    ctr = (f(T0 + h) - f(T0 - h)) / (2 * h)
    print(f"  h = {h:.0e}: forward rel.err = {abs(fwd / df_exact(T0) - 1):.1e},"
          f" central rel.err = {abs(ctr / df_exact(T0) - 1):.1e}")
f_np = lambda T: 1.0 / (np.exp(1.0 / T) - 1.0)  # numpy version accepts complex input
cs = np.imag(f_np(T0 + 1e-30j)) / 1e-30  # complex step: no subtractive cancellation
print(f"  complex step (h = 1e-30) rel.err = {abs(cs / df_exact(T0) - 1):.1e}")
check_grads(f, (T0,), order=2, modes=["fwd", "rev"])  # raises if AD disagrees with FD
print("  check_grads: passed (1st and 2nd order, forward and reverse mode)")


# --- Part B: an event threshold inside a fixed-step time loop (toy) ---
# L(t) = exp(theta * t); a wind switches on when L > L_c and removes mass at a
```

```python
# fixed rate. Mass lost by time t_end:  Delta M = rate * (t_end - t_s),
# with event time t_s = ln(L_c) / theta.
L_c, rate, t_end, n_steps = 10.0, 1.0, 5.0, 2000
dt = t_end / n_steps


def mass_lost_loop(theta):
    def body(i, dm):
        t = i * dt
        wind_on = jnp.exp(theta * t) > L_c  # boolean: no derivative flows through it
        return dm + jnp.where(wind_on, rate * dt, 0.0)
    return jax.lax.fori_loop(0, n_steps, body, 0.0)


def mass_lost_event(theta):  # locate the event, then differentiate through t_s(theta)
    t_s = jnp.log(L_c) / theta
    return rate * (t_end - t_s)


th = 1.0
exact = rate * jnp.log(L_c) / th**2
h = 1e-2  # much wider than one stair (theta changes by ~dt*theta^2/ln(L_c) ~ 1e-3 per stair)
fd = (mass_lost_loop(th + h) - mass_lost_loop(th - h)) / (2 * h)
print("\nPart B: d(Delta M)/d(theta) at theta = 1")
print(f"  exact (continuum)       = {exact:.4f}")
print(f"  jax.grad through loop   = {jax.grad(mass_lost_loop)(th):.4f}   <-- wrong")
print(f"  central FD, h = 1e-2    = {fd:.4f}")
print(f"  jax.grad, event-aware   = {jax.grad(mass_lost_event)(th):.4f}")
```

Output (executed for these notes):

```
Part A: d/dT [1/(exp(1/T)-1)] at T = 0.3
  exact      = 4.262471213033907e-01
  jax.grad   = 4.262471213033907e-01
  h = 1e-02: forward rel.err = 2.6e-02, central rel.err = 3.4e-04
  h = 1e-05: forward rel.err = 2.6e-05, central rel.err = 3.3e-10
  h = 1e-08: forward rel.err = 2.5e-08, central rel.err = 1.7e-09
  h = 1e-11: forward rel.err = 1.4e-06, central rel.err = 5.5e-07
  complex step (h = 1e-30) rel.err = 0.0e+00
  check_grads: passed (1st and 2nd order, forward and reverse mode)

Part B: d(Delta M)/d(theta) at theta = 1
  exact (continuum)       = 2.3026
  jax.grad through loop   = 0.0000   <-- wrong
  central FD, h = 1e-2    = 2.3750
  jax.grad, event-aware   = 2.3026
```

Single precision

In 32-bit arithmetic $\epsilon_{\text{mach}} \approx 1.2 \times 10^{-7}$, so even the best central difference is accurate only to about 10^{-5}, and an ill-conditioned linear solve can lose every digit. Do derivative checks, Newton solves and Fisher matrices in 64-bit.

5.4 When AD is confidently wrong

Events inside a fixed-step solver. Part B of Listing 9 models a wind that switches on when $L(t) = e^{\theta t}$ crosses a threshold L_c. The continuum mass loss $\Delta M = \dot{M}\,(t_{\text{end}} - t_s)$ depends on θ through the event time $t_s = \ln L_c/\theta$, so $\mathrm{d}\Delta M/\mathrm{d}\theta = \dot{M} \ln L_c/\theta^2 = 2.30$. The fixed-step loop

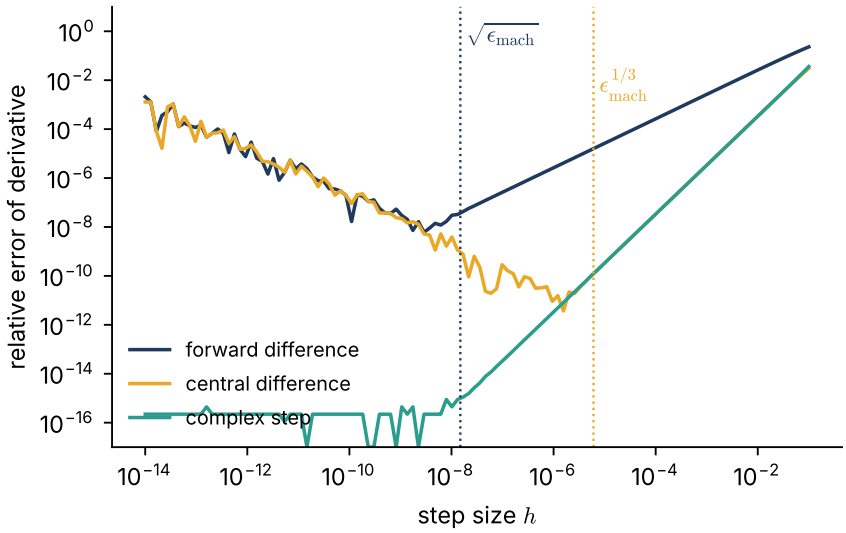


Figure 8: Relative error of three numerical derivatives of $1/(e^{1/T}-1)$ at $T=0.3$ against step size. Each finite difference has a V-shaped error with its minimum at the predicted step (dotted lines); the complex step is exact to rounding for every small h.

evaluates $\texttt{where}(L > L_c, \dot{M}\,\Delta t, 0)$ at each step. For fixed θ the boolean has no derivative, and the rate does not depend on θ, so AD returns exactly **zero**. It is the correct derivative of the program, which is a staircase in θ (Fig. 9); it is not the derivative of the physics. A finite difference with h large compared with the stair width recovers the slope approximately (2.38; the residual error is the quantization of the staircase). Locating the event and differentiating $t_s(\theta)$ restores the exact answer.

This is the generic failure for hybrid systems: phase transitions in stellar evolution, close encounters and collisions in N-body integration, any adaptive scheme whose control flow depends on the state. The derivative through a fixed execution path omits how the event time, the jump at the event, and any change of topology depend on the parameters. Remedies are event location with implicit differentiation of the event time, saltation matrices that propagate sensitivities across a jump, and solver-level event handling (for example in Diffrax).

In Jaxstro

Startrax phase switches, Stellax helium ignition and convective-boundary changes, and Gravax close encounters are all events. A gradient-based fit or HMC run that crosses one with a path-wise derivative will see a flat or misleading landscape there. Check gradients by finite differences with h larger than the solver's event resolution on both sides of each known transition.

Discontinuities inside an expectation are the second failure, treated next.

Exercises for this lecture

Exercises 18, 19, numbered as in Appendix A of the full notes. Worked solutions are in Appendix B; attempt each one before reading its solution.

18. [P] Implicit and adjoint derivatives. For $R(s,\theta) = s^3 + s - \theta = 0$: find $\mathrm{d}s/\mathrm{d}\theta$ at $\theta = 2$ by

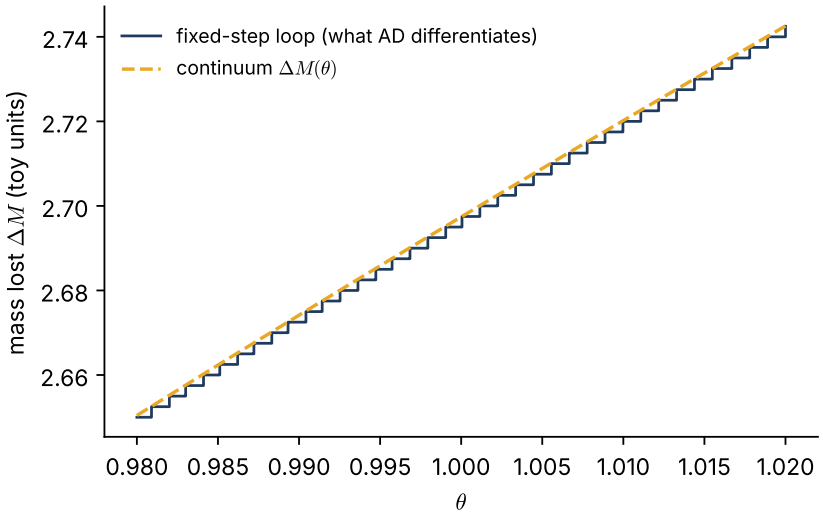


Figure 9: The program AD differentiates is a staircase whose slope is zero almost everywhere; the physical quantity is the smooth dashed line.

the implicit function theorem, then $\mathrm{d}\mathcal{L}/\mathrm{d}\theta$ for $\mathcal{L} = \frac{1}{2}s^2$ by the adjoint equation. Verify against the explicit solution near $\theta = 2$. Contrast with $R = s^2 - \theta$ as $\theta \to 0^+$.

19. **[C] Make the event differentiable.** Rewrite `mass_lost_event` in Listing 9 so that it does *not* use the closed form $t_s = \ln L_c/\theta$: locate the event numerically with a derivative-free root finder and obtain its derivative with `jax.lax.custom_root`. Confirm the gradient to 10 digits.

References

[1] A. G. Baydin, B. A. Pearlmutter, A. A. Radul, and J. M. Siskind. Automatic differentiation in machine learning: a survey. *Journal of Machine Learning Research*, 18(153):1–43, 2018.

[2] M. Blondel, Q. Berthet, M. Cuturi, R. Frostig, S. Hoyer, F. Llinares-López, F. Pedregosa, and J.-P. Vert. Efficient and modular implicit differentiation. In *Advances in Neural Information Processing Systems*, volume 35, 2022.

[3] W. Squire and G. Trapp. Using complex variables to estimate derivatives of real functions. *SIAM Review*, 40(1):110–112, 1998.